

WEEK-5

Task-1

Implement a producer-consumer communication system using anonymous pipes where the parent process generates data and the child process consumes it. Measure the communication efficiency.

CODE:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <time.h>
#define NUM_ITEMS 100000
int main()
{
    int pipefd[2];
    pid_t pid;
    if (pipe(pipefd) == -1)
    {
        perror("pipe");
        return 1;
    }
    pid = fork();

    if (pid < 0)
    {
        perror("fork");
        return 1;
    }
```

```

/* Child Process - Consumer */
if (pid == 0)
{
    close(pipefd[1]);

    int data;

    long long sum = 0;

    int count = 0;

    while (read(pipefd[0], &data, sizeof(data)) > 0)
    {
        sum += data;

        count++;
    }

    close(pipefd[0]);

    printf("\n--- Consumer (Child) ---\n");

    printf("Items consumed : %d\n", count);

    printf("Sum of data   : %lld\n", sum);

    exit(0);
}

/* Parent Process - Producer */
else
{
    close(pipefd[0]);

    int data;

    clock_t start, end;

    start = clock();

    for (int i = 1; i <= NUM_ITEMS; i++)
    {
        data = i;
    }
}

```

```

        if (write(pipefd[1], &data, sizeof(data)) == -1)
        {
            perror("write");
            close(pipefd[1]);
            return 1;
        }
    }
    close(pipefd[1]);
    wait(NULL);
    end = clock();
    double time_taken =
        (double)(end - start) / CLOCKS_PER_SEC;
    double data_transferred =
        (double)(NUM_ITEMS * sizeof(int)) / (1024 * 1024);
    double throughput =
        data_transferred / time_taken;
    printf("\n--- Producer (Parent) ---\n");
    printf("Items produced    : %d\n", NUM_ITEMS);
    printf("Data transferred    : %.2f MB\n", data_transferred);
    printf("Communication time : %.6f seconds\n", time_taken);
    printf("Throughput        : %.2f MB/s\n", throughput);
}
return 0;
}

```

```

root@LAPTOP-RJFD8EBT:~# nano producer_consumer.c
root@LAPTOP-RJFD8EBT:~# ls -l producer_consumer.c
-rw-r--r-- 1 root root 1899 Sep 16 06:53 producer_consumer.c
root@LAPTOP-RJFD8EBT:~# ls -l producer_consumer.c
-rw-r--r-- 1 root root 1899 Sep 16 06:53 producer_consumer.c
root@LAPTOP-RJFD8EBT:~# gcc producer_consumer.c -o producer_consumer
root@LAPTOP-RJFD8EBT:~# gcc producer_consumer.c -o producer_consumer
root@LAPTOP-RJFD8EBT:~# ./producer_consumer

--- Consumer (Child) ---
Items consumed : 100000
Sum of data : 5000050000

--- Producer (Parent) ---
Items produced : 100000
Data transferred : 0.38 MB
Communication time : 0.007729 seconds
Throughput : 4.34 MB/s
root@LAPTOP-RJFD8EBT:~# ps -f
UID      PID     PPID  C STIME TTY          TIME CMD
root      286     283  0 06:52 pts/0    00:00:00 -bash
root     1465     286  50 06:55 pts/0    00:00:00 ps -f
root@LAPTOP-RJFD8EBT:~# ps -f
UID      PID     PPID  C STIME TTY          TIME CMD
root      286     283  0 06:52 pts/0    00:00:00 -bash
root     1500     286  0 06:55 pts/0    00:00:00 ps -f
root@LAPTOP-RJFD8EBT:~# ^C
root@LAPTOP-RJFD8EBT:~# strace -f -e tracepipe,clone,fork,wait4,close ./producer_consumer
close(3)           = 0
clone(child_stack=NULL, flags=CLONE_CHILD_CLEARPID|CLONE_CHILD_SETTID|SIGCHLD, strace: Process
  1915 attached
  child_tidptr=0x716b98fela10) = 1915
[pid 1914] close(3) = 0
[pid 1915] close(4) = 0
[pid 1914] close(4) = 0
[pid 1914] wait(-1, <unfinished ...>) = 0
[pid 1915] close(3) = 0

--- Consumer (Child) ---
Items consumed : 100000

--- Producer (Parent) ---
Items produced : 100000
Data transferred : 0.38 MB
Communication time : 2.988618 seconds
Throughput : 0.13 MB/s
+++ exited with 0 +++
root@LAPTOP-RJFD8EBT:~# pstree -p
systemd(1)─agetty(197)
└─cron(164)
   └─dbus-daemon(167)
      └─init-systemd(Ub(2))─SessionLeader(280)─Relay(286)(283)─bash(286)─pstree+
         └─init(7)─{init}(8)
            └─login(290)─bash(371)
               └─{init-systemd(Ub)(9)}
                  └─rsyslogd(180)─{rsyslogd}(199)
                     └─{rsyslogd}(280)
                        └─{rsyslogd}(281)
                           └─systemd(349)─(sd-pam)(350)
                              └─systemd-journal(56)
                                 └─systemd-logind(174)
                                    └─systemd-resolve(150)
                                       └─systemd-timesyn(152)─{systemd-timesyn}(161)
                                          └─systemd-udev(104)
                                             └─unattended-upgr(198)─{unattended-upgr}(247)
                                                └─wsl-pro-service(2200)─{wsl-pro-service}(2210)
                                                   └─{wsl-pro-service}(2211)
                                                      └─{wsl-pro-service}(2212)
                                                         └─{wsl-pro-service}(2213)
                                                            └─{wsl-pro-service}(2214)
root@LAPTOP-RJFD8EBT:~#

```

TASK-2:

Develop a program that executes the equivalent of the shell command: `ls -l grep` using `fork()`, `pipe()`, `dup2()`, and `exec()` system calls

Code:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <unistd.h>
```

```
#include <sys/wait.h>
```

```
int main()
```

```
{
```

```
    int pipefd[2];
```

```
    pid_t pid1, pid2;
```

```
    /* Create pipe */
```

```
    if (pipe(pipefd) == -1)
```

```
    {
```

```
        perror("pipe");
```

```
        return 1;
```

```
    }
```

```
    /* Create first child */
```

```

pid1 = fork();
if (pid1 < 0)
{
    perror("fork");
    return 1;
}
/* First child executes ls -l */
if (pid1 == 0)
{
    close(pipefd[0]);
    /* Redirect stdout to pipe */
    dup2(pipefd[1], STDOUT_FILENO);
    close(pipefd[1]);
    execlp("ls", "ls", "-l", NULL);
    perror("execlp ls");
    exit(1);
}
/* Create second child */
pid2 = fork();
if (pid2 < 0)
{
    perror("fork");
    return 1;
}
/* Second child executes grep c */
if (pid2 == 0)
{
    close(pipefd[1]);
    /* Redirect stdin from pipe */

```

[illegible]

```

[pid 488] close(4) = 0
[pid 488] close(4) = 0
[pid 488] close(4) = 0
[pid 488] close(4) = 0
[pid 488] close(4) = 0
[pid 488] close(4) = 0
[pid 488] close(4) = 0
[pid 488] close(3) = 0
[pid 488] close(3) = 0
[pid 488] close(1) = 0
[pid 488] close(2-rwxr-xr-x 1 root root 16576 Aug 17 05:59 command_executor
)
= 0
-rw-r--r-- 1 root root 1252 Aug 17 05:59 command_executor.c
-rwxr-xr-x 1 root root 16216 Aug 29 10:07 copy
-rw-r--r-- 1 root root 727 Aug 29 09:48 copy.c
-rw-r--r-- 1 root root 727 Aug 29 10:07 copy.c
[pid 488] +++ exited with 0 +++
[pid 487] <... wait4 resumed>NULL, 0, NULL) = 488
[pid 487] --- SIGCHLD {si_signo=SIGCHLD, si_code=CLD_EXITED, si_pid=488, si_uid=0, si_status=0, si_etime=0, si_stime=4 /*
0.04 s */} ---
-rwxr-xr-x 1 root root 16184 Aug 29 09:52 copyfile
[pid 487] wait4(489, -rw-r--r-- 1 root root 944 Aug 29 09:51 copyfile.c
-rwxr-xr-x 1 root root 16304 Aug 18 03:51 fork_process
-rw-r--r-- 1 root root 1178 Aug 18 03:50 fork_process.c
-rw-r--r-- 1 root root 1367 Sep 16 08:08 ls_grep.c
-rwxr-xr-x 1 root root 16216 Aug 19 18:44 process
-rw-r--r-- 1 root root 736 Aug 19 18:39 process.c
-rw-r--r-- 1 root root 850 Aug 19 18:21 process_demo.c
-rwxr-xr-x 1 root root 16264 Aug 18 04:12 process_exp
-rw-r--r-- 1 root root 973 Aug 18 04:11 process_exp.c
-rwxr-xr-x 1 root root 16512 Aug 19 18:25 process_states
-rw-r--r-- 1 root root 1685 Aug 19 18:24 process_states.c
-rw-r----- 1 root root 0 Aug 18 04:10 process_states.c.save
-rwxr-xr-x 1 root root 16440 Sep 16 06:54 producer_consumer
-rw-r--r-- 1 root root 1899 Sep 16 06:53 producer_consumer.c
-rw-r--r-- 1 root root 64 Aug 29 10:07 source.txt
-rw-r--r-- 1 root root 555 Aug 12 06:44 task1.c
-rw-r--r-- 1 root root 1048 Aug 12 06:55 task2.c
-rw-r--r-- 1 root root 1715 Aug 12 07:01 task3.c
-rw-r--r-- 1 root root 4219 Aug 12 07:10 task4.c
-rw-r--r-- 1 root root 3086 Aug 12 08:10 task5.c
-rw-r--r-- 1 root root 8129 Aug 29 10:17 trace.txt
<unfinished ...>
[pid 489] close(1) = 0
[pid 489] close(2) = 0
[pid 489] +++ exited with 0 +++
<... wait4 resumed>NULL, 0, NULL) = 489
--- SIGCHLD {si_signo=SIGCHLD, si_code=CLD_EXITED, si_pid=489, si_uid=0, si_status=0, si_etime=0, si_stime=1 /* 0.01 s */}
---

Command equivalent to: ls -l | grep c
Execution completed successfully.
+++ exited with 0 +++

```